

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo A (azul).
Profesor: Isabel Pita.

Nombre del alumno:

1. Dadas las siguientes funciones de coste, indica su orden de complejidad (representado por la función más sencilla) y ordena los órdenes de complejidad obtenidos utilizando $\subset$.

- a) $f_1(n) = 6$
- b) $f_2(n) = 4n + \log n^2$
- c) $f_3(n) = n^5 + 3n^3 + 2$
- d) $f_4(n) = 2^n + 3^n$
- e) $f_7(n) = 3n \log 5n + \log n$

2. Dadas dos matrices m_1 y m_2 de dimensiones $m_1 \times m_2$ y $m_2 \times m_3$, indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justificala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
for (int i = 0; i < m1; ++i) {  
    for (int j = 0; j < m3; ++j) {  
        int suma = 0;  
        for (int k = 0; k < m2; ++k)  
            suma += m1[i][k] * m2[k][j]  
        p[i][j] = suma;  
    }  
}
```

3. Escribe un predicado para expresar que todos los valores de un vector v son diferentes.
4. Escribe un predicado que dado un vector v y dos índices del vector i, j , indique si entre los valores correspondientes a los índices del vector existe una inversión. Entre dos valores existe una inversión si el primero es mayor que el segundo.
5. Especifica un algoritmo que dado un vector de números enteros, todos ellos diferentes, cuya longitud es mayor que cero calcule el número de inversiones del vector.
6. Dada la especificación:

P: $\{v.size() \geq 0\}$

resolver(vector<int> v) dev int np

Q: $\{np == \max p, q : 0 \leq p \leq q \leq v.size() \wedge secPositiva(v, p, q) : q - p\}$

Y dada la siguiente implementación:

```
int resolver(std::vector<int> const& v) {  
    int np = 0; int longAct = 0;  
    for (int i = 0; i < v.size(); ++i) {  
        if (v[i] > 0) { // El elemento continua la racha  
            ++longAct;  
            if (np < longAct) np = longAct;  
        }  
        else longAct = 0; // Se rompe la racha  
    }  
    return np;  
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo. Recuerda que en el invariante se debe expresar lo que se calcula en todas las variables que intervienen en el bucle, no solo en la variable que se devuelve como resultado de la función.

7. Dado un vector de números enteros cuyos elementos son todos diferentes, y no están ordenados, indica un algoritmo que permita obtener un valor del vector que no sea ni el máximo ni el mínimo en tiempo constante.
8. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado e indica el coste final del algoritmo.

```
lli f(std::vector<int> & v, int ini, int fin) {
    if (fin <= ini) return 0;
    else {
        int m = (ini + fin) / 2;
        lli iz = f(v, ini, m);
        lli dr = f(v, m+1, fin);
        lli aux = 0;
        int i = ini, j = m+1;
        while (i <= m && j <= fin) {
            if (v[i] > v[j]) {
                aux += (m-i+1);
                ++j;
            }
            else ++i;
        }
        std::inplace_merge(v.begin(), v.begin()+m, v.begin()+fin);
        return iz + dr + aux;
    }
}
```

9. Escribe una función recursiva que permita resolver el problema de comprobar si en un vector algún valor coincide con su posición en el vector.
10. Indica cuál es la mejor complejidad que puede tener un algoritmo de ordenación basado en comparaciones, si los valores del vector son números enteros cualesquiera. Indica si conoces algún algoritmo con esta complejidad y en caso afirmativo dí cuál o cuales son.